

--{ python для пентестера }--

Ethical Hacking Tutorial Group The Codeby

представляет



PYTHON

«Для Пентестера»

Programming Language..



-- { ПРОДВИНУТЫЙ УРОВЕНЬ } --

Наследование

Наследование позволяет создавать новый класс на основе уже существующего класса. Этот класс называется производный класс (**derived class**) или дочерний (**child class**). Подкласс унаследует от суперкласса - базового класса (**base class**) все публичные атрибуты и методы. Базовый класс называют также родительским (**parent class**).

Общая схема наследования:

```
class Суперкласс:  
    bla-bla-bla
```

```
class Подкласс(Суперкласс):  
    атрибуты подкласса
```

Например у нас кроме пользователей сайта есть суперпользователь root, у которого есть те же самые данные – login, password, email. Но это юзер операционной системы, поэтому размещать его данные вместе с пользователями сайта нелогично. Что мы можем сделать?

Вариант 1: просто создать новый класс со всеми методами и объектами. Этот вариант плохой – много строк одного и того же кода.

Вариант 2: использовать наследование класса.

Как это сделать?

Создадим ещё один класс Root, и укажем в скобках имя предыдущего класса **Root(Users)**. Таким образом, созданный нами класс станет дочерним (ребёнком или подклассом) родительского класса (суперкласса) Users(), и может использовать все атрибуты и методы этого класса. На скриншоте ниже видно, что класс Root имеет заглушку pass, и прекрасно работает. При этом мы написали всего 2 строки кода в классе.

```

class Users:
    def __init__(self, login, password, email):
        self.login = login
        self.password = password
        self.email = email

class Root(Users):
    pass

admin = Users('admin', '12345', 'superadmin@gmail.com')
testuser = Users('testuser', 'pass_test', 'test@site.com')
root = Root('root', 'toor', 'root@localhost')
print(admin.login, admin.password, admin.email)
print(testuser.login, testuser.password, testuser.email)
print(root.login, root.password, root.email)

```

test x

```

admin 12345 superadmin@gmail.com
testuser pass_test test@site.com
root toor root@localhost

```

Мы можем создавать для пользователя root любые объекты и методы в собственном классе, которые будут недоступны родительскому классу.

```

admin = Users('admin', '12345', 'superadmin@gmail.com')
testuser = Users('testuser', 'pass_test', 'test@site.com')
root = Root('root', 'toor', 'root@localhost')
root.ssh_pass = 'S14SUtp1v3QR'
print('Пароль SSH:', root.ssh_pass)

```

test x

```

Пароль SSH: S14SUtp1v3QR

```

Мы можем переписать код класса Root, без использования заглушки pass, используя метод `__init__`.

```

class Users:
    def __init__(self, login, password, email):
        self.login = login
        self.password = password
        self.email = email

class Root(Users):
    def __init__(self, login, password, email):
        super().__init__(login, password, email)

admin = Users('admin', '12345', 'superadmin@gmail.com')
testuser = Users('testuser', 'pass_test', 'test@site.com')
root = Root('root', 'toor', 'root@localhost')
root.ssh_pass = 'S14SUtp1v3QR'
print('Пароль SSH:', root.ssh_pass)

```

test x

Пароль SSH: S14SUtp1v3QR

А что это за конструкция? `super().__init__()`

По умолчанию, подкласс унаследует методы из родительского класса, но подкласс может переопределять (override) методы родительского класса.

Смысл заключается в том, что Python автоматически не запустит родительский конструктор, поскольку мы его переопределили в классе Root. Поэтому методом `super()` мы вызываем родительский конструктор.

Говоря по-другому, метод `super()` вызывает родительский метод в дочерний и использует его.

Обратите внимание – с методом `super()` параметр `self` в скобках писать НЕ надо!

Аналогичная запись будет с явным названием родительского класса.

```

class Root(Users):
    def __init__(self, login, password, email):
        Users.__init__(self, login, password, email)

```

Мы можем создать, например, ключевой аргумент.

```

class Root(Users):
    def __init__(self, login, password, email, ssh_pass='S14SUp1v3QR'):
        self.ssh_pass = ssh_pass ←
        Users.__init__(self, login, password, email)

admin = Users('admin', '12345', 'superadmin@gmail.com')
testuser = Users('testuser', 'pass_test', 'test@site.com')
root = Root('root', 'toor', 'root@localhost')
print('Пароль SSH:', root.ssh_pass)

```

test ×

Пароль SSH: S14SUp1v3QR

Ещё один вариант того же кода, в 3-х решениях одновременно.

```

class Users:
    def __init__(self, login, password, email):
        self.login = login
        self.password = password
        self.email = email

class Root(Users):
    def __init__(self, bla_bla): ←
        self.bla_bla = bla_bla
        super().__init__(login='root', password='toor', email='root@localhost')

admin = Users('admin', '12345', 'superadmin@gmail.com')
testuser = Users('testuser', 'pass test', 'test@site.com')
root = Root('Здесь всё что угодно')
root.ssh_pass = 'S14SUp1v3QR'
print('Пароль SSH:', root.ssh_pass, '\n' + root.bla_bla, '\n' + root.login)

```

1 ×

Пароль SSH: S14SUp1v3QR
Здесь всё что угодно
root

Полиморфизм

Полиморфизм в объектно-ориентированном программировании – это возможность обработки данных принадлежащих к разным классам, с помощью одинакового имени функции, или метода. При этом, результаты работы одноименных методов могут существенно различаться.

Кстати на картинке выше мы уже наблюдали полиморфизм между классами, связанными наследованием. У каждого есть метод `__init__()`, но результат разный.

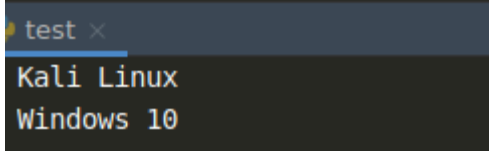
Однако классы не обязательно должны быть связаны наследованием. Простейший пример – две одинаковых функции `os`, но вывод получаем разный. В данном примере используется декоратор `@staticmethod`.

Статический метод - это метод, который ничего не знает о классе или экземпляре. `@staticmethod` означает, что когда этот метод вызывается, мы не передаем ему экземпляра класса, а значит, что нет необходимости передавать неявный аргумент, например `self`.

```
class Hacker:
    @staticmethod
    def os():
        print('Kali Linux')

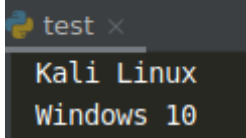
class Victim:
    @staticmethod
    def os():
        print('Windows 10')

os = Hacker()
os.os()
system = Victim()
system.os()
```



Доступ к таким методам можно получить как из экземпляра класса, так и из самого класса.

```
Hacker.os()
Victim.os()
```



Ещё один декоратор `@classmethod`

При вызове этого метода мы передаем класс (`cls`) как первый аргумент вместо экземпляра этого класса (как обычно мы делаем с методами). Это означает, что вы можете использовать класс и его свойства внутри этого метода, а не конкретный экземпляр.

```
class Hacker:
    @classmethod
    def os(cls):
        print('Kali Linux')

class Victim:
    @classmethod
    def os(cls):
        print('Windows 10')

Hacker.os()
Victim.os()
```

test x

Kali Linux

Windows 10

Декораторы

Декоратор — это функция, которая позволяет обернуть другую функцию для расширения её функциональности без непосредственного изменения её кода. Или если сказать по-другому, то декоратор - это функция, ожидающая ДРУГУЮ функцию в качестве параметра.

Допустим у нас есть простой класс Exr который принимает 2 аргумента и возводит одно число в степень другого числа.

```
class Exp:
    def __init__(self, x, y):
        self.x = x
        self.y = y

    def res(self):
        return self.x**self.y

times = Exp(10, 100)
print(times.res())
```


То есть в этом примере весь код будет выглядеть так:

```
def timer(fun):
    import time

    def speed(*args):
        start = time.time()
        result = fun(*args)
        print("Время выполнения функции: %f" % (time.time()-start))
        return result

    return speed
```

```
@timer
class Exp:
    def __init__(self, x, y):
        self.x = x
        self.y = y

    def res(self):
        return self.x**self.y

times = Exp(10, 100)
print(times.res())
```

[illegible]

Таким образом мы можем применять какую-нибудь нам нужную функцию без использования импорта пользовательского модуля. А также применять созданный декоратор в совершенно разных программах, декорируя всего лишь одной строкой кода.

Главное, что нужно понять - это то, что декоратор состоит из двух вложенных друг в друга функций. Первая - сам декоратор (в аргументах он принимает функцию или метод для декорирования), вторая - обёртка - эта функция принимает аргументы переданного объекта. Соответственно внутри второй функции - обёртки, у нас в распоряжении будет и исходная функция/метод, и его аргументы. Не забывайте использовать оператор `return` для возврата данных из декоратора - ведь декорируемый объект так же может возвращать какие-то значения.